



cropping tool

source

destination

google cloud run function

by Ritvik Maini



proposal

logic

demonstration



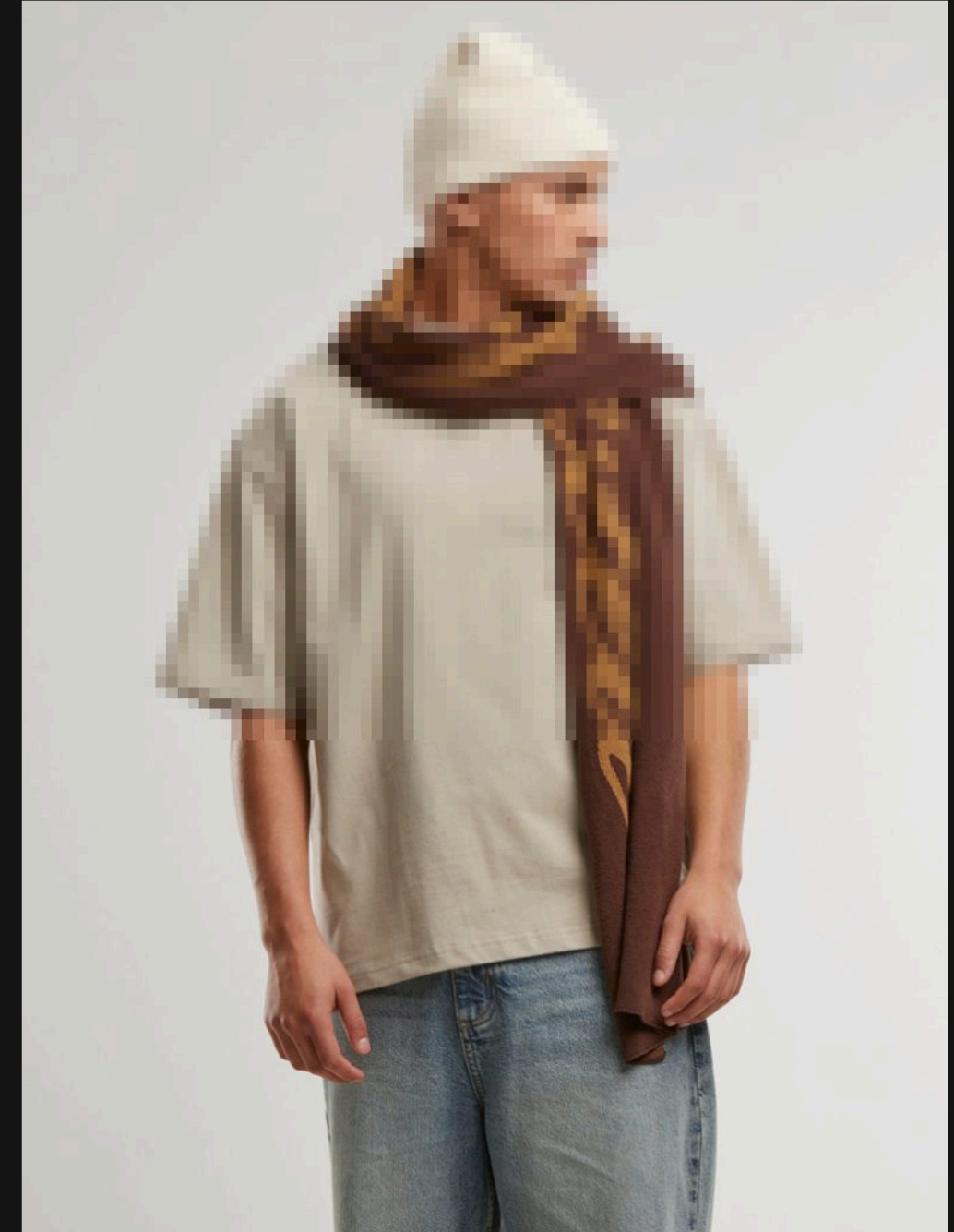
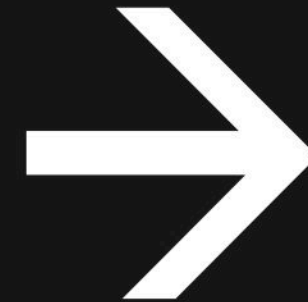
next steps

TOP HALF

source

destination

google cloud run function



BOTTOM HALF

source

destination

google cloud run function

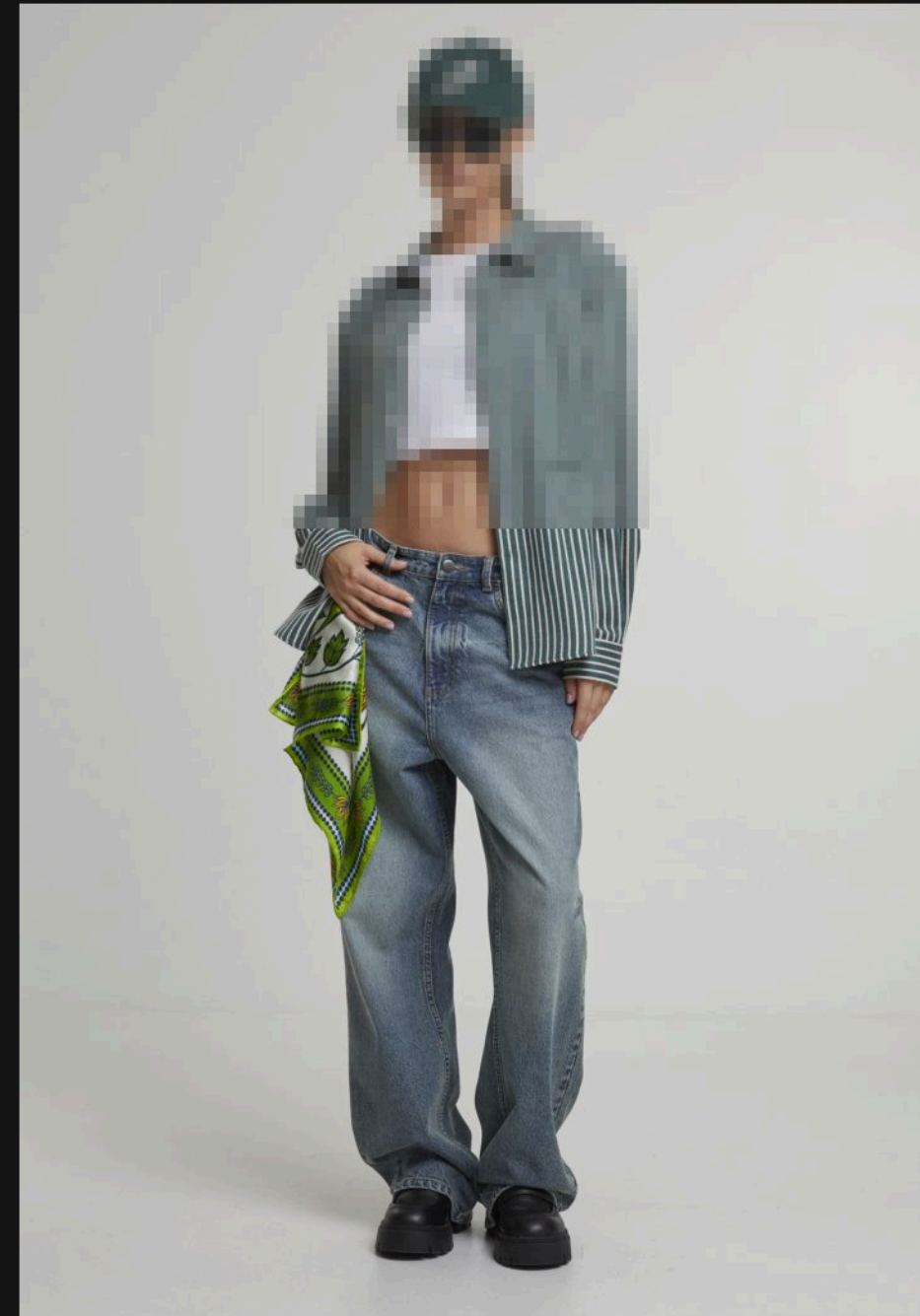
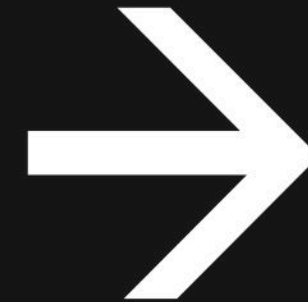
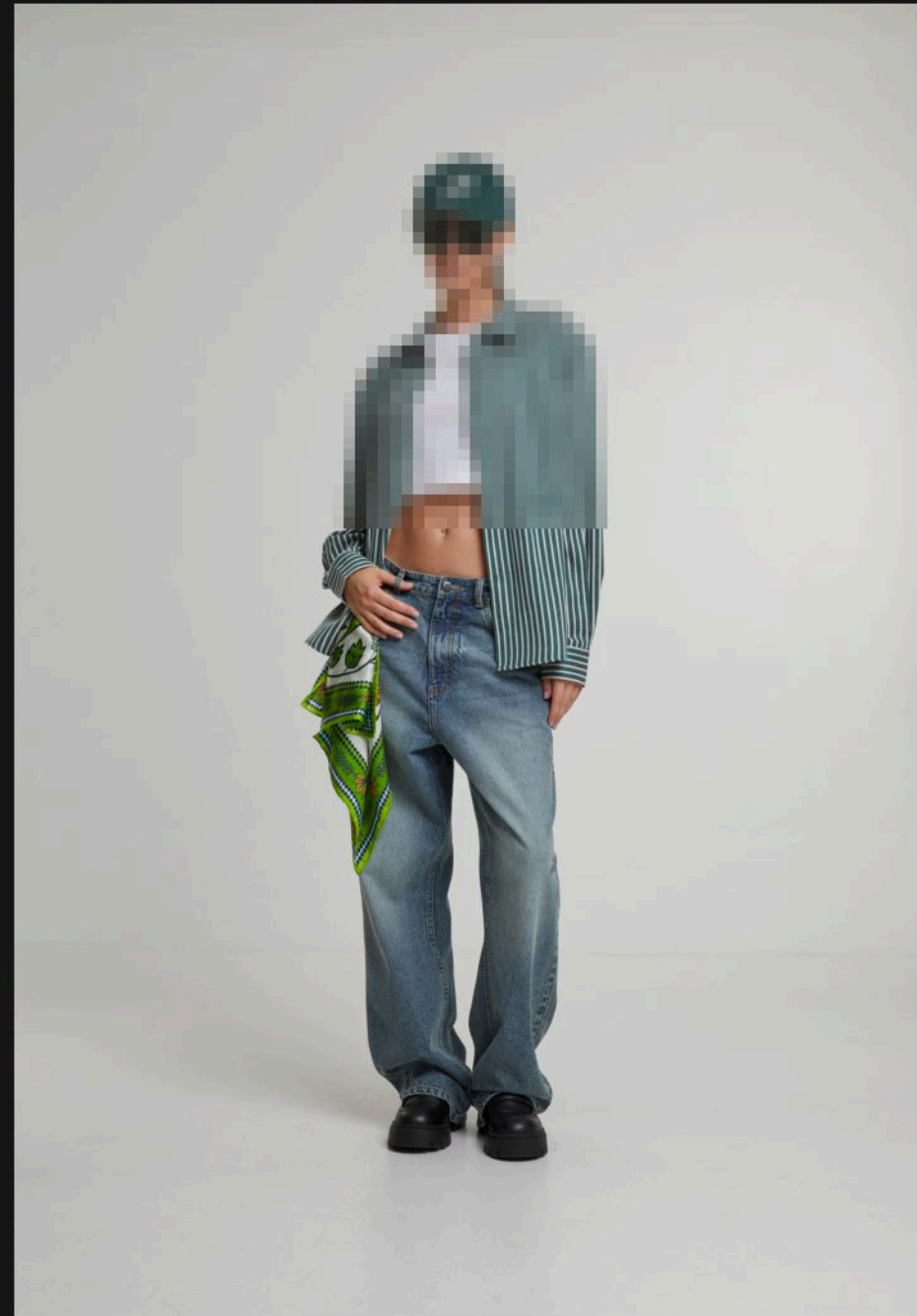


FULL CROP

source

destination

google cloud run function

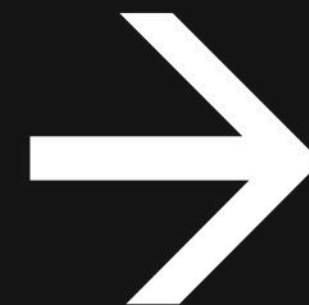


PRODUCT*

source

destination

google cloud run function

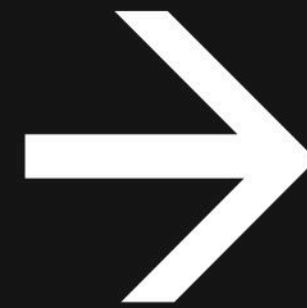


PRODUCT*

source

destination

google cloud run function





proposal

logic

demonstration



next steps

01 SETUP →

Source and Destination
Storage Buckets

gle Cloud Run
ction

source
destination
google cloud run function

pose_croper_gcf



01 SETUP



source

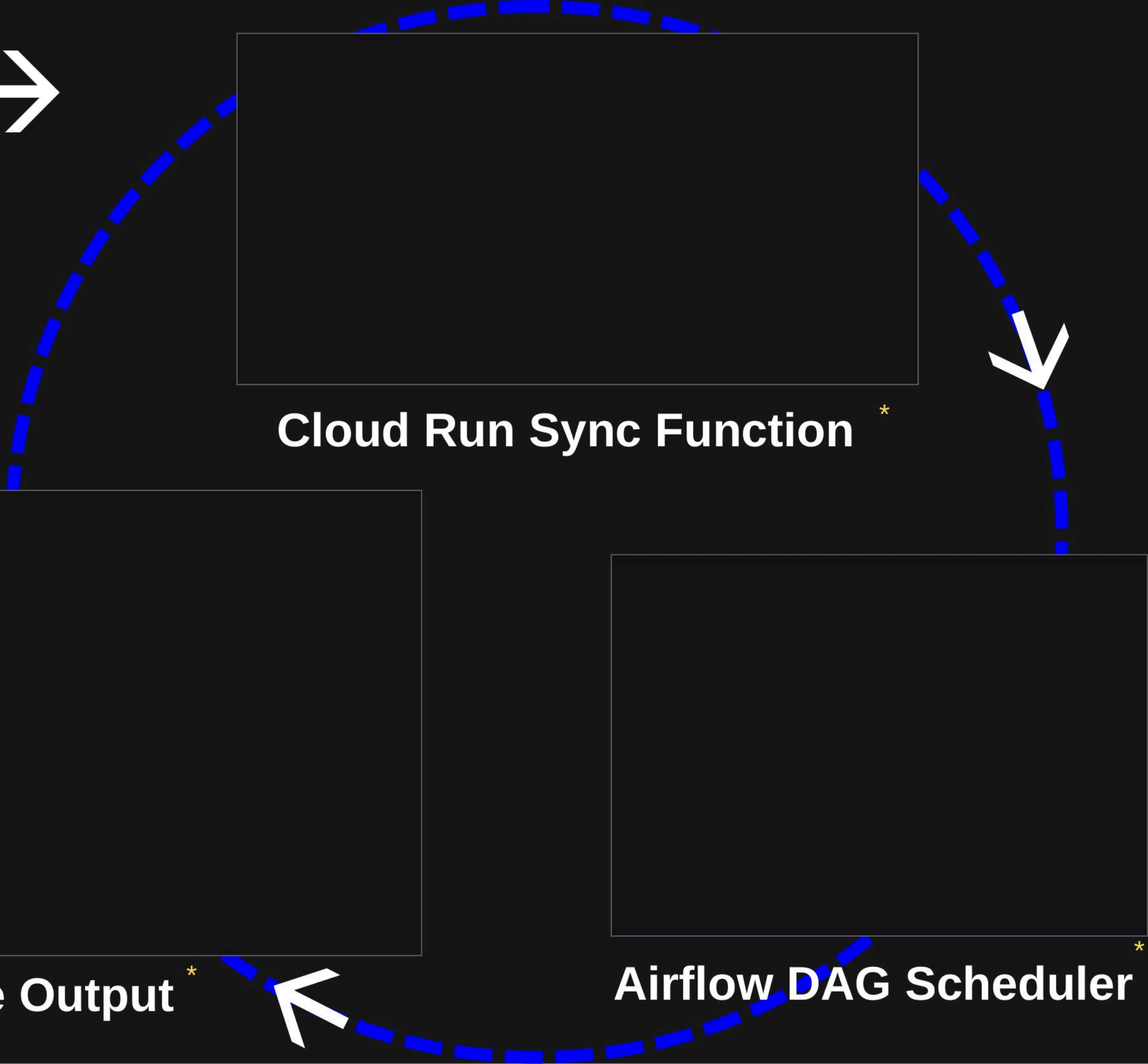
destination

google cloud run function

Google Drive Output

Cloud Run Sync Function

Airflow DAG Scheduler



02 IMAGE PROCESSING

1

BigQuery

To ensure accurate image processing, we leverage data from the Naming Convention to identify the article group and assign the appropriate crop.

Images that follow the naming convention are processed accordingly, while those that do not are managed using default values, to ensure that non conform images can also be processed.

catalog lookup: StyleCode -> EAN fallback across two catalog views

source

destination

google cloud run function

02 IMAGE PROCESSING

2

Segmentation Methods *

Product images identified by `_P*` are processed by detecting the largest bounding box using Otsu's Binarization, K-Means Clustering or Grabcut Segmentation. If no valid bounding box is found (covering 12%-97% of the image area), the entire image is used as a fallback.

The cropping logic centers around the detected bounding box, maintaining an aspect ratio of 1.33 (height:width). To enhance framing, 10% padding is applied around the box, ensuring balanced and visually aligned outputs while keeping the crop within image boundaries.



Otsu's
Binarization



K-Means
Segmentation



Grabcut
Segmentation



Fallback
Method



Bounding Box
Validation

Method Used: Otsu Binarization
Initial Bounding Box: [42, 865, 1336, 1460]
Adjusted Bounding Box: [42, 302, 1336, 2022]
Crop Area: 79.62%
Aspect Ratio: 1.33
Padding Ratio: 0



source

destination

google cloud run function

02 IMAGE PROCESSING

3 Pose from Google's Mediapipe

MediaPipe Pose is a machine learning framework used to detect and annotate human body landmarks from images.

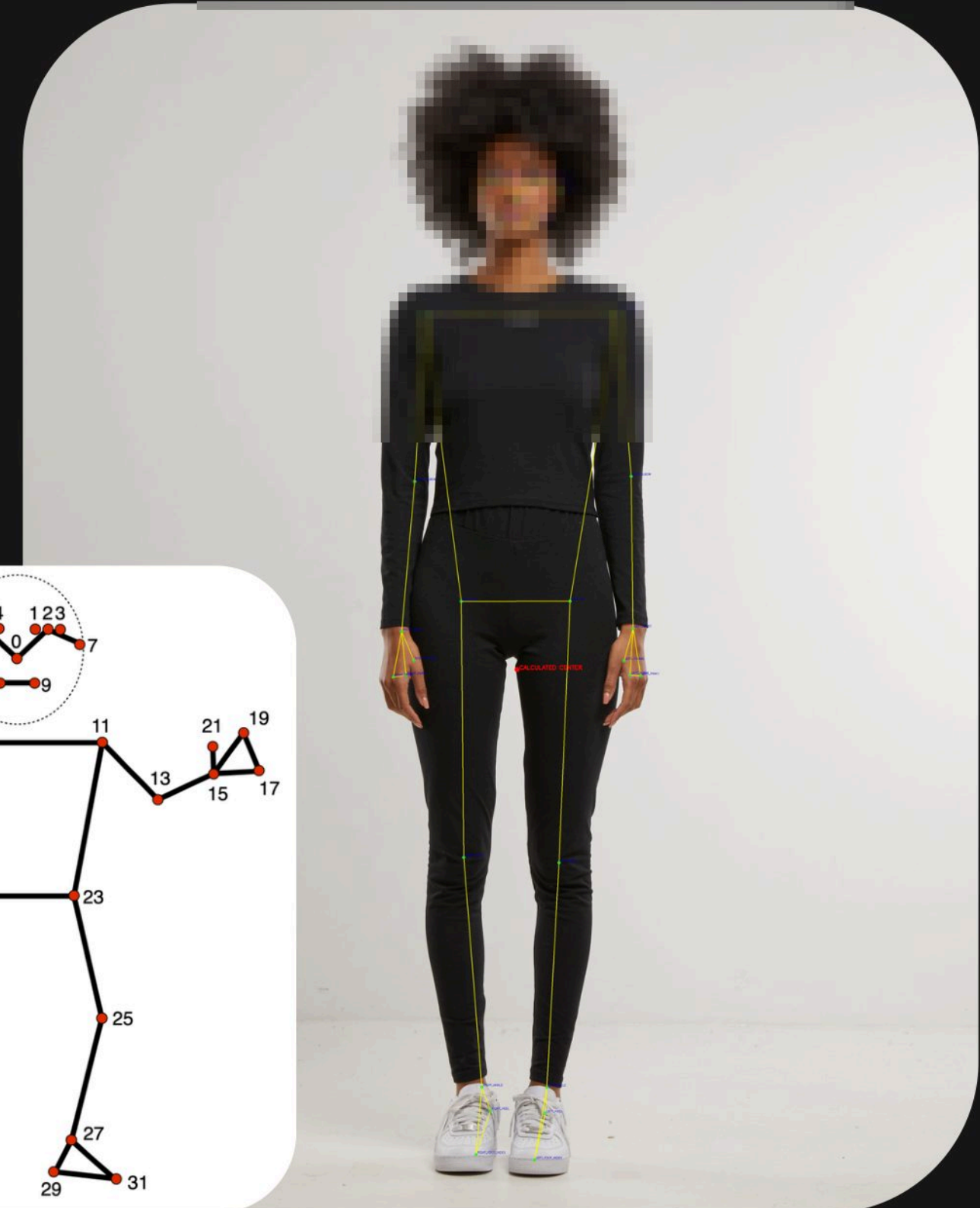
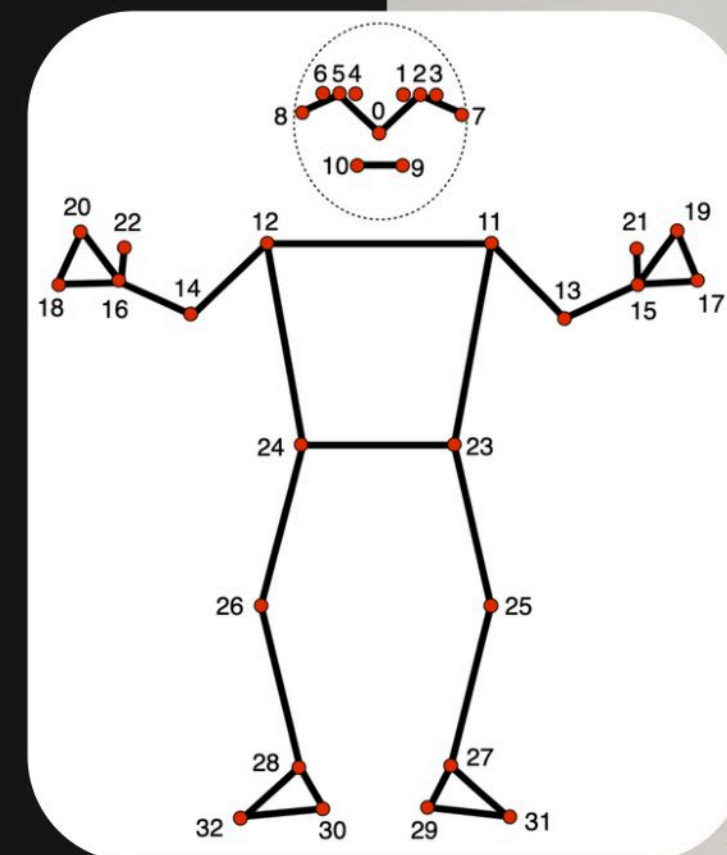
It processes images to identify 32 key points on the body, the detected extremities are used by this tool to find a center around which the cropping logic orients itself

To maintain proper framing, extra space is added around the landmarks—about 12% above and 7% below—while additional padding of 30% above and 20% below the center ensures balanced and visually aligned cropped outputs, all while preserving an aspect ratio of 1.33.

source

destination

google cloud run function



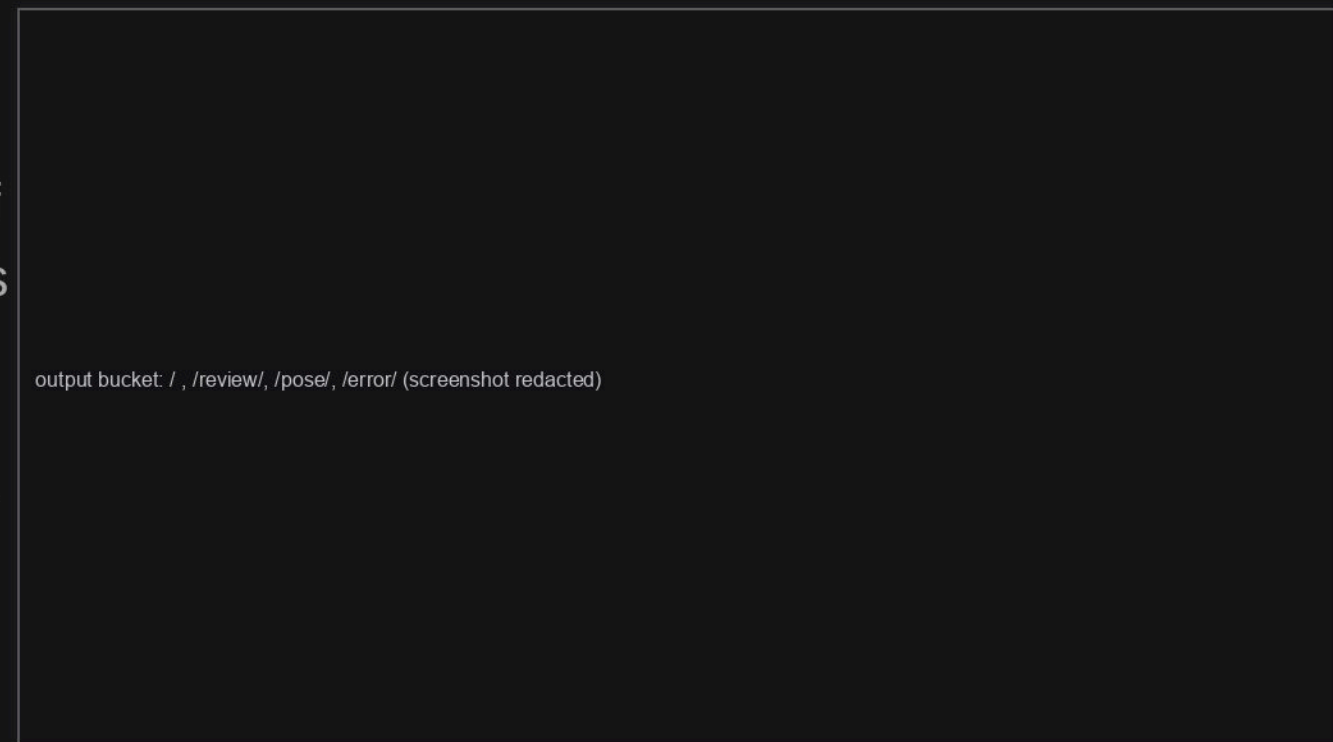
03 OUTPUT STRUCTURE

The `pose_cropper_gcf` function processes images by classifying them with an article group and attempting reliable cropping.

If successful, it generates a single output image and saves it in the base folder. However, if the image cannot be confidently classified or requires further review, the function generates all three crop variants and saves them in a dedicated review folder. This ensures that unreliable results are flagged.

Additionally, the function supports advanced visualization outputs, product segmentation or landmark-annotated images, which are saved separately in a `pose` folder. These outputs help visualize the detected features without modifying the original cropping logic.

In case of any errors during processing, the original image is saved to an `error` folder for clear communication. This structured pipeline ensures accurate results while maintaining flexibility for handling edge cases or errors.



source

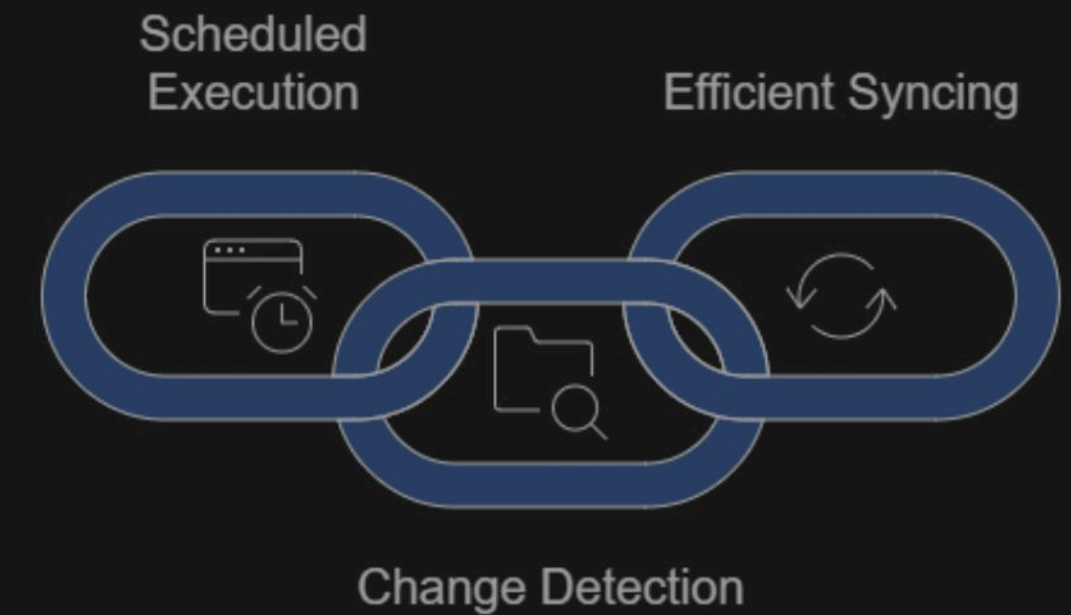
destination

google cloud run function

04 SYNC →

The second Cloud Run Function '*pose_picture_gdrive_folder_sync*' efficiently syncs .jpg files from the Destination Storage Bucket to Google Drive by performing incremental synchronization. It runs on a scheduled basis (every 7 minutes), first comparing top-level folders and the number of files in them between Bucket and Drive to detect changes. When differences are found—such as missing files or folders—it uploads the missing files, creating any necessary folders in Drive to mirror the Bucket structure. The script also handles subfolders recursively, ensuring that nested folder structures are synced if needed.

The script uses multi-threading with `ThreadPoolExecutor` and 100 workers to handle uploads in parallel, ensuring efficiency even for larger changes, while also including error handling to retry only problematic files, minimizing resource usage under normal conditions and triggering heavier syncing only when updates are required.



source

destination

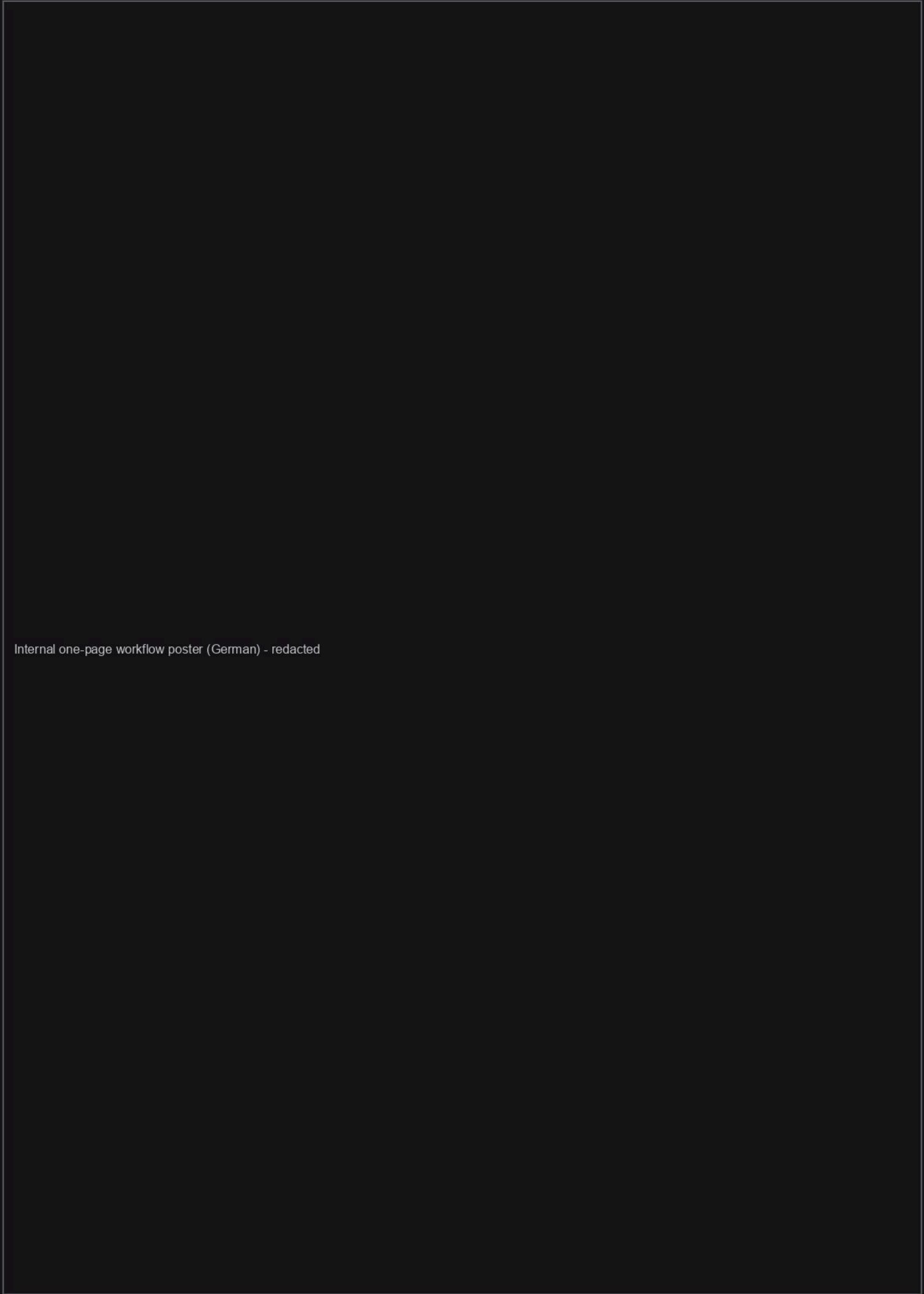
google cloud run function

POSTER

source

destination

google cloud run function

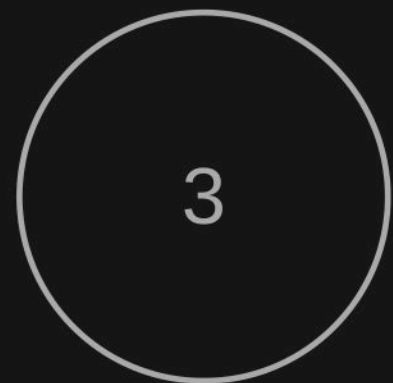




proposal

logic

demonstration



next steps

ANY THOUGHTS?

source

destination

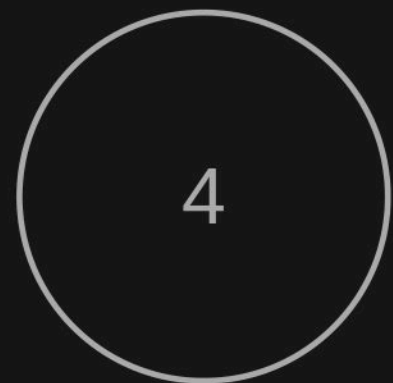
google cloud run function



proposal

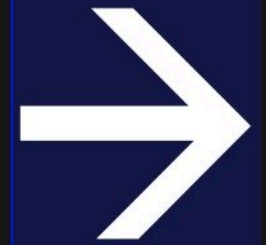
logic

demonstration



next steps

OPEN TO SUGGESTIONS



**MODEL IDENTIFICATION AND BUILDING IT
AS AN ATTRIBUTE**

source

destination

google cloud run function



THANK
YOU!

source

destination

google cloud run function